Todo App Architecture

Two Docker containers: an Nginx UI and an Express API backed by SQLite.

Browser	UI container - Nginx	Backend container - Node.js	Persistent volume
User opens localhost:8081	Serves public/ files Proxies /api/* to backend:3000	Express JSON API Validates requests Informative request and todo logs sql.js reads and writes SQLite	/app/data todo-data volume Retains todos across restarts

Request flow

The browser calls the same-origin /api routes. Nginx forwards those requests over the private Compose network to the backend service. Express handles validation and CRUD operations, then returns JSON. The backend writes database changes to its mounted volume.

API routes

Route	Action
GET /api/todos	List todos
POST /api/todos	Create todo from a title
PATCH /api/todos/:id	Set completed state
DELETE /api/todos/:id	Delete todo
GET /api/health	Backend health check

Run and operate

Start both containers with docker compose up --build -d. Open http://localhost:8081. The API is also exposed at port 3000. View backend logs with docker compose logs -f backend. Stop containers with docker compose down; the named volume keeps data.